

```
Initialize attributes of the parent class.
Then initialize attributes specific to an electric car.
"""
super().__init__(make, model, year)
self.battery = Battery()
```

Now we can make a new file called *my_electric_car.py*, import the `ElectricCar` class, and make an electric car:

```
my_electric    from car import ElectricCar
_car.py
my_leaf = ElectricCar('nissan', 'leaf', 2024)
print(my_leaf.get_descriptive_name())
my_leaf.battery.describe_battery()
my_leaf.battery.get_range()
```

This has the same output we saw earlier, even though most of the logic is hidden away in a module:

```
2024 Nissan Leaf
This car has a 40-kWh battery.
This car can go about 150 miles on a full charge.
```

Importing Multiple Classes from a Module

You can import as many classes as you need into a program file. If we want to make a regular car and an electric car in the same file, we need to import both classes, `Car` and `ElectricCar`:

```
my_cars.py ❶ from car import Car, ElectricCar

❷ my_mustang = Car('ford', 'mustang', 2024)
print(my_mustang.get_descriptive_name())
❸ my_leaf = ElectricCar('nissan', 'leaf', 2024)
print(my_leaf.get_descriptive_name())
```

You import multiple classes from a module by separating each class with a comma ❶. Once you've imported the necessary classes, you're free to make as many instances of each class as you need.

In this example we make a gas-powered Ford Mustang ❷ and then an electric Nissan Leaf ❸:

```
2024 Ford Mustang
2024 Nissan Leaf
```

Importing an Entire Module

You can also import an entire module and then access the classes you need using dot notation. This approach is simple and results in code that is easy to read. Because every call that creates an instance of a class includes the module name, you won't have naming conflicts with any names used in the current file.